JAVA NOTES — DATA TYPES & CASTING

10-page study notes • Primitive types, conversions, promotion and arithmetic

1. Overview of Java primitive types

Java has 8 primitive data types. They represent simple values and are not objects.

Two numeric families

- *Integral: byte, short, int, long and char — whole-number values.*
- *Floating-point: float and double — approximate real-number values.*
- *Logical: boolean — only true or false; it is not numeric.*

```
PRIMITIVE TYPES
├── Integral: byte, short, int, long, char
├── Floating-point: float, double
└── Logical: boolean
```

Example declarations

```
int age = 21;
double price = 99.50;
char grade = 'A';
boolean passed = true;
```

Remember: The type of a variable determines which values it can store and which operations are valid.

2. INTEGRAL TYPES — byte, short, int, long

Java language fundamentals • Handwritten-style revision notes

Whole-number types

Integral types store whole numbers without a fractional part. Java defines fixed ranges for its four signed integral types.

Type	Bits	Minimum	Maximum
byte	8	−128	127
short	16	−32,768	32,767
int	32	-2^{31}	$2^{31}-1$
long	64	-2^{63}	$2^{63}-1$

When to use each

- byte: compact data and byte-oriented APIs.
- short: smaller-range integer; uncommon in everyday calculations.
- int: usual default for whole-number calculations.
- long: values that may exceed the int range.

```
byte level = 100;
short year = 2026;
int population = 1_200_000;
long views = 9_000_000_000L;
```

Remember: An unsuffixed integer literal is normally int if it fits. Use L (prefer uppercase) for a long literal.

3. FLOAT, DOUBLE, CHAR & BOOLEAN

Java language fundamentals • Handwritten-style revision notes

Floating-point types

- *float: 32-bit IEEE 754; roughly 6–7 significant decimal digits.*
- *double: 64-bit IEEE 754; roughly 15–16 significant decimal digits. Default type for decimal literals.*

```
float temperature = 36.5f; // f suffix
double pi = 3.141592653589793;
double big = 1.25e6;          // 1,250,000
```

char

`char` *is a 16-bit unsigned UTF-16 code unit. It stores one code unit, not necessarily one complete Unicode character.*

```
char initial = 'R';
char symbol = '\u03A9'; // Greek omega
String greeting = "Hello";
```

boolean

A boolean stores only `true` *or* `false`*. It is used in conditions and logical expressions.*

```
boolean active = true;
boolean eligible = age >= 18;
if (eligible && active) {
    System.out.println("Allowed");
}
```

- *char uses single quotes; String uses double quotes.*
- *boolean cannot be converted to or from a number in Java.*

Remember: Floating-point values are approximate. For example, 0.1 + 0.2 may not be exactly 0.3.

4. WIDENING & NARROWING CONVERSIONS

Java language fundamentals • Handwritten-style revision notes

Widening primitive conversion

Widening converts a value to a type with a broader range. It is usually automatic.

```
byte → short → int → long → float → double
            char → int → long → float → double
```

- *Widening normally does not throw an exception.*
- *A large long converted to float or double can still lose precision.*

```
int count = 120;
long total = count;       // automatic widening
double result = total;    // automatic widening
```

Narrowing primitive conversion

Narrowing converts to a type with a smaller range or precision. Java generally requires an explicit cast.

```
double price = 19.99;
int whole = (int) price; // 19; fraction removed

long big = 130L;
byte small = (byte) big; // -126
```

- *Floating-point to integral conversion truncates toward zero.*
- *Narrowing may lose high-order bits or precision.*

Remember: A cast requests a conversion; it does not guarantee that the original value will fit.

5. IMPLICIT & EXPLICIT CASTING

Java language fundamentals • Handwritten-style revision notes

Implicit conversion

Java performs an implicit conversion automatically when the language permits it, commonly for widening conversions.

```
int marks = 85;
double score = marks; // int becomes double
System.out.println(score); // 85.0
```

Explicit conversion (casting)

Write the target type in parentheses before the value or expression.

```
double average = 87.9;
int whole = (int) average; // 87

int code = 65;
char letter = (char) code; // 'A'
```

Casting does not round

- *Casting 8.99 to int gives 8, not 9.*
- *For rounding, use methods such as Math.round() where appropriate.*

```
double x = 8.99;
int a = (int) x;        // 8
long b = Math.round(x); // 9
```

Constant expression assignment exception

A compile-time constant int expression can be assigned to byte, short or char without a cast if its value fits.

```
byte a = 100;     // allowed
byte b = 10 + 20;  // allowed
int n = 100;
// byte c = n;     // error: variable int
byte d = (byte) n;
```

Remember: Parsing text is different from casting: use Integer.parseInt("123") to convert numeric text to int.

6. OVERFLOW & UNDERFLOW

Java language fundamentals • Handwritten-style revision notes

Integer overflow

An integer overflow occurs when an arithmetic result is outside the type's range. Java integer arithmetic wraps according to fixed-width two's-complement representation; it does not automatically throw an overflow exception.

```
int max = Integer.MAX_VALUE;
System.out.println(max);    // 2147483647
System.out.println(max+1);  // -2147483648
```

Integer underflow

Going below the minimum signed integer value wraps to the maximum.

```
int min = Integer.MIN_VALUE;
System.out.println(min - 1); // 2147483647
```

Floating-point overflow and underflow

- *Floating-point overflow can produce positive or negative Infinity.*
- *Floating-point underflow can produce a subnormal value or zero.*

```
double huge = 1e308 * 1e10; // Infinity
double tiny = 1e-300 * 1e-50; // may become 0.0
```

Avoiding surprises

- *Choose a wider type when the expected range is large.*
- *Validate values before arithmetic when necessary.*
- *Use Math.addExact(), subtractExact() or multiplyExact() for checked int/long arithmetic.*

```
int safe = Math.addExact(10, 20);
// Math.addExact(Integer.MAX_VALUE, 1)
// throws ArithmeticException
```

Remember: Integer overflow wraps; floating-point overflow can produce Infinity.

7. EXPRESSION TYPE PROMOTION — PART 1

Java language fundamentals • Handwritten-style revision notes

Unary numeric promotion

Before many unary numeric operations, byte, short and char values are promoted to int.

```
byte b = 10;
int answer = -b; // b promoted to int
char ch = 'A';
int next = ch + 1; // 66
```

Binary numeric promotion

For arithmetic with two numeric operands, Java promotes them to a common type. The result type follows the promoted type.

Promotion priority

- *If either operand is double, both are converted to double.*
- *Otherwise, if either is float, both become float.*
- *Otherwise, if either is long, both become long.*
- *Otherwise, both operands become int.*

```
byte a = 10, b = 20;
int sum = a + b; // result is int

int x = 5;
long y = 10L;
long z = x + y; // result is long
```

Mixed numeric expression

```
int count = 4;
double price = 2.5;
double total = count * price; // 10.0
```

Remember: Even byte + byte, short + short or char + char normally produces an int result.

8. EXPRESSION TYPE PROMOTION — PART 2

Java language fundamentals • Handwritten-style revision notes

More promotion examples

```
byte a = 2, b = 3;
// byte c = a + b; // compile-time error
int c = a + b;     // correct

char letter = 'A';
int value = letter + 2; // 67

long id = 100L;
float rate = 1.5f;
float result = id * rate; // float
```

Division and remainder

The result type depends on operand promotion. Integer division discards the fractional part; floating-point division keeps it.

```
int a = 7 / 2;       // 3
double b = 7 / 2;    // 3.0 (int division first)
double c = 7 / 2.0;  // 3.5
int rem = 7 % 2;     // 1
```

Boolean expressions are different

Relational and equality operators produce boolean results. Logical operators work with boolean values, not numeric values.

```
int age = 20;
boolean adult = age >= 18;
boolean allowed = adult && true;
```

Evaluation order

Parentheses make grouping clear. Operator precedence determines grouping, while promotion determines operand types.

```
double result = (2 + 3) * 1.5; // 7.5
```

Remember: Assigning an expression to double does not retroactively make earlier integer arithmetic floating-point arithmetic.

9. COMPOUND ASSIGNMENT CONVERSION

Java language fundamentals • Handwritten-style revision notes

What is compound assignment?

*Compound assignment combines an operation with assignment. Examples: +=, -=, *=, /= and %=*

```
int score = 10;
score += 5; // similar to score = score + 5
score *= 2; // similar to score = score * 2
```

Special implicit narrowing

A compound assignment includes an implicit conversion back to the left-hand variable's type. The equivalent simple assignment may not compile.

```
byte n = 10;
// n = n + 1; // error: expression is int
n += 1;        // allowed; converted back to byte
```

Conceptually, `n += 1` *behaves like* `n = (byte)(n + 1)`*, with the left-hand expression evaluated only once.*

Data loss can still happen

```
byte value = 127;
value += 1;
System.out.println(value); // -128
```

- *Compound assignment does not protect against overflow.*
- *It does not guarantee that a value safely fits the destination type.*
- *The conversion rule also applies to *=, /=, -= and other compound operators.*

Remember: Use compound assignment for concise updates, but check the destination range when narrowing or overflow is possible.

10. QUICK REVISION & PRACTICE

Java language fundamentals • Handwritten-style revision notes

One-page memory sheet

- *Eight primitive types: byte, short, int, long, float, double, char, boolean.*
- *Integral: byte, short, int, long, char. Floating-point: float and double.*
- *Decimal literal defaults to double; add f for float and L for long.*
- *Widening is usually automatic; narrowing generally needs a cast.*
- *Casting floating-point to integer truncates toward zero.*
- *Integer overflow wraps; floating-point overflow may become Infinity.*
- *byte, short and char promote to int for most arithmetic.*
- *Binary promotion priority: double → float → long → int.*
- *Compound assignment converts back to the left-hand type.*

Practice questions

- *1. What is the default type of 4.5? Answer: double.*
- *2. What is (int) 9.8? Answer: 9.*
- *3. What is the type of byte + byte? Answer: int.*
- *4. What is Integer.MAX_VALUE + 1? Answer: wraps to Integer.MIN_VALUE.*
- *5. Why does byte b=2; b+=1; compile? Answer: compound assignment converts back to byte.*
- *6. Can boolean be cast to int? Answer: No.*

Predict the output

```
System.out.println(5 / 2);      // ?
System.out.println(5 / 2.0);    // ?
byte x = 127;
x += 1;
System.out.println(x);          // ?
char c = 'A';
System.out.println(c + 1);      // ?
```

Answers: 2, 2.5, -128, 66. Compile each example and experiment with the values.